

Estudo e Análise de Transformers

Relatório da Aula 5

Matheus Costa Monteiro

Introdução

Os Transformers representam uma das arquiteturas mais importantes da história recente do Processamento de Linguagem Natural. Introduzidos por Vaswani et al. em 2017, esses modelos substituíram mecanismos recorrentes por mecanismos de atenção, permitindo maior paralelismo no treinamento e melhor modelagem de dependências de longo alcance. Este relatório responde algumas perguntas sobre a arquitetura Transformer, abordando seus principais componentes, seu treinamento, a função dos embeddings e uma breve comparação entre dois modelos amplamente utilizados baseados nessa família.

1. Leitura do artigo *Attention Is All You Need*

O artigo *Attention Is All You Need*, de Vaswani et al., propõe uma nova arquitetura baseada exclusivamente em mecanismos de atenção, eliminando a necessidade das então usadas, RNN's para tarefas de modelagem de sequência. A principal contribuição do trabalho é mostrar que a atenção, quando combinada com projeções lineares e codificação posicional, é suficiente para alcançar resultados de ponta em tarefas como tradução automática.

A arquitetura proposta é composta por um codificador (*encoder*) e um decodificador (*decoder*), ambos formados por blocos empilhados. Cada bloco do codificador contém um

mecanismo de *multi-head self-attention* seguido por uma rede feed-forward. Já cada bloco do decodificador inclui *masked self-attention*, atenção sobre a saída do codificador e uma rede feed-forward. Em todos os blocos são utilizadas conexões residuais e normalização.

O trabalho também introduz o mecanismo de *scaled dot-product attention*, a atenção multi-cabeça e o uso de *positional encoding* senoidal para incorporar informação de ordem na sequência. O ganho central da arquitetura está na sua capacidade de capturar relações globais entre tokens com maior eficiência computacional do que modelos recorrentes tradicionais.

2. Leitura do Capítulo 8 de Jurafsky e Martin

O Capítulo 8 do livro de Jurafsky e Martin apresenta os Transformers como a arquitetura dominante em PLN moderno. O capítulo reforça a intuição de que a atenção permite que cada token interaja diretamente com todos os demais tokens da sequência, diferentemente de RNNs, em que a informação precisa ser propagada passo a passo.

Além disso, o capítulo destaca a distinção entre arquiteturas apenas codificadoras, apenas decodificadoras e do tipo codificador-decodificador. Essa separação é importante porque diferentes modelos são adaptados para diferentes tarefas. Modelos como BERT utilizam apenas o codificador e são fortemente orientados para compreensão de texto. Modelos como GPT utilizam apenas o decodificador e são voltados para geração autoregressiva. Já modelos como T5 e o Transformer original utilizam a arquitetura completa para tarefas *sequence-to-sequence*.

O capítulo também detalha o papel dos embeddings, da *masked attention*, do treinamento com *teacher forcing*, da predição do próximo token e da importância de pré-treinamento em larga escala.

3. Execução e análise do código do Transformer

Foi analisado o material *The Annotated Transformer*, que reimplementa o modelo original de forma didática. Esse código é particularmente útil porque mostra de maneira modular a construção dos componentes fundamentais do Transformer, como:

- embeddings de entrada;
- codificação posicional;
- mecanismo de atenção;
- *multi-head-attention*;
- redes feed-forward;

- camadas de normalização e conexões residuais;
- procedimento de treinamento.

A principal vantagem do código anotado é deixar explícito como os tensores são transformados ao longo do pipeline. Em particular, observa-se que:

1. os tokens são convertidos em vetores densos por meio de embeddings;
2. esses vetores recebem informação posicional;
3. a atenção calcula interações entre todos os tokens da sequência;
4. múltiplas cabeças de atenção aprendem diferentes tipos de relação;
5. a saída final é projetada para o vocabulário para viabilizar a predição do próximo token.

Do ponto de vista pedagógico, esse código é excelente para entender a arquitetura, embora não seja otimizado para produção. Em aplicações reais, bibliotecas como PyTorch, TensorFlow e Hugging Face abstraem grande parte dessa implementação.

4. O que são K, Q e V e quais são suas funções

No mecanismo de atenção, cada token da sequência é projetado linearmente em três representações distintas:

- **Q (Query)**: representa o que o token atual busca nos demais tokens;
- **K (Key)**: representa quais características cada token oferece para serem consultadas;
- **V (Value)**: representa o conteúdo efetivo que será agregado ao resultado final da atenção.

A intuição é a seguinte: cada *query* é comparada com todas as *keys* para medir similaridade. Essas similaridades geram pesos de atenção. Em seguida, os pesos são usados para fazer uma combinação ponderada dos *values*.

Formalmente, dados os tensores Q , K e V , a atenção é calculada como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

onde d_k é a dimensionalidade das chaves. Assim, Q e K determinam onde olhar, enquanto V determina o que extrair.

5. O que é *scaled dot-product attention*

O *scaled dot-product attention* é o mecanismo central do Transformer. Ele calcula a relevância entre tokens por meio do produto escalar entre *queries* e *keys*. Esse produto indica o quanto um token deve prestar atenção a outro.

O procedimento ocorre em três etapas:

1. calcular QK^T , obtendo a similaridade entre cada par de tokens;
2. dividir o resultado por $\sqrt{d_k}$, para evitar que valores grandes prejudiquem a estabilidade numérica do *softmax*;
3. aplicar *softmax* e multiplicar por V , produzindo uma combinação ponderada dos valores.

O termo *scaled* é essencial, pois o produto escalar tende a crescer em magnitude conforme a dimensionalidade aumenta. Sem esse fator de escala, o *softmax* poderia saturar, dificultando o treinamento.

6. O que é e como funciona o *positional encoding*

Como o Transformer não possui recorrência nem convolução, ele não tem um mecanismo nativo para perceber a ordem dos tokens. Para resolver isso, a arquitetura incorpora informação posicional aos embeddings de entrada por meio do *positional encoding*.

No artigo original, esse encoding é definido usando funções seno e cosseno de diferentes frequências:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

onde pos representa a posição do token e i representa a dimensão do vetor.

As principais vantagens dessa formulação são:

- permitir que o modelo diferencie tokens com base na posição;
- possibilitar a generalização para sequências maiores do que as observadas no treinamento;
- fornecer uma forma contínua e determinística de codificar ordem.

Na prática, o vetor posicional é somado ao embedding do token antes de entrar nas camadas de atenção.

7. Como é feito o treinamento de um Transformer

O treinamento de um Transformer depende da tarefa e do tipo de arquitetura utilizada, mas o processo geral envolve os seguintes elementos:

7.1 Entrada e embeddings

Os tokens da sequência de entrada são convertidos em embeddings densos. Em seguida, soma-se o *positional encoding* para inserir informação de ordem.

7.2 Propagação direta

A sequência é processada por blocos de atenção e redes feed-forward. No caso de modelos autoregressivos, utiliza-se uma máscara causal para impedir que o modelo veja tokens futuros.

7.3 Objetivo de treinamento

Em tarefas de linguagem, o objetivo mais comum é prever o próximo token da sequência. Em arquiteturas do tipo BERT, o treinamento geralmente usa objetivos como *masked language modeling*. Já em modelos do tipo T5, a formulação é mais geral, tratando diferentes tarefas como problemas de geração de texto.

7.4 Cálculo da perda

A saída do modelo é projetada no espaço do vocabulário, gerando distribuições de probabilidade. A perda normalmente é calculada com entropia cruzada entre a distribuição prevista e o token alvo.

7.5 Backpropagation e otimização

Após o cálculo da perda, usa-se retropropagação para atualizar os parâmetros do modelo. O artigo original utiliza o otimizador Adam com uma estratégia específica de aquecimento da taxa de aprendizado (*learning rate warmup*), seguida de decaimento.

7.6 Treinamento em larga escala

Na prática moderna, Transformers são treinados em grandes volumes de texto, com lotes extensos, paralelismo em múltiplas GPUs e técnicas adicionais de regularização, normalização, mistura de precisão e particionamento de parâmetros.

8. Como são os embeddings de um Transformer

Os embeddings em Transformers são representações vetoriais densas associadas aos tokens do vocabulário. Cada token é mapeado para um vetor de dimensão fixa d_{model} . Esses embeddings são aprendidos durante o treinamento.

Em geral, os embeddings possuem as seguintes características:

- são densos e contínuos;
- são treináveis;
- têm a mesma dimensionalidade do modelo;
- podem ser somados ao *positional encoding*;
- em muitos modelos, a matriz de embeddings de entrada compartilha pesos com a camada de saída.

É importante distinguir embeddings estáticos de embeddings contextuais. Em modelos mais antigos, como Word2Vec e GloVe que implementamos nas aulas passadas, cada palavra possui um vetor fixo. Já em Transformers, o embedding inicial do token é apenas o ponto de partida. Após passar pelas camadas do modelo, a representação final torna-se contextual, ou seja, depende dos demais tokens da sequência.

9. Comparação entre dois Transformers

Para esta atividade, foi realizada uma comparação conceitual entre dois modelos muito conhecidos: **BERT** e **T5**.

9.1 BERT

O BERT (*Bidirectional Encoder Representations from Transformers*) é um modelo baseado apenas no codificador do Transformer. Seu principal diferencial é o processamento bidirecional do contexto, o que o torna muito forte em tarefas de compreensão, como classificação, inferência textual e resposta a perguntas.

Vantagens do BERT:

- excelente desempenho em tarefas discriminativas;
- forte capacidade de representação contextual;
- arquitetura eficiente para *fine-tuning* em tarefas supervisionadas.

Limitações do BERT:

- não foi desenhado primariamente para geração livre de texto;

- depende de cabeças específicas para diferentes tarefas.

9.2 T5

O T5 (*Text-To-Text Transfer Transformer*) utiliza a arquitetura completa de codificador-decodificador e reformula todas as tarefas como problemas de texto para texto. Assim, tradução, sumarização, classificação e outras tarefas podem ser tratadas com o mesmo paradigma.

Vantagens do T5:

- grande flexibilidade de uso;
- unificação de múltiplas tarefas no mesmo formato;
- excelente desempenho em tarefas generativas e *sequence-to-sequence*.

Limitações do T5:

- maior custo computacional;
- inferência mais pesada que modelos apenas codificadores.

9.3 Comparação geral

De forma geral, o BERT tende a ser mais apropriado para tarefas de compreensão e classificação, enquanto o T5 é mais versátil e se destaca quando a tarefa pode ser formulada naturalmente como transformação de texto.

Conclusão

O estudo realizado mostra que os Transformers consolidaram uma mudança estrutural no PLN ao substituir recorrência por mecanismos de atenção. Os conceitos de Q , K e V , a atenção escalada por produto escalar e a codificação posicional formam a base matemática da arquitetura. Além disso, o treinamento em larga escala e o uso de embeddings contextuais explicam boa parte do sucesso desses modelos.

A análise do código do *Annotated Transformer* foi importante para compreender a implementação prática dos principais componentes do modelo. Por fim, a comparação entre BERT e T5 evidencia que, embora ambos pertençam à mesma família, suas estruturas e objetivos os tornam mais adequados para diferentes tipos de aplicação.

Referências

- [1] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems, 2017.
- [2] JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3. ed. Online manuscript, 2026. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>.
- [3] HUANG, A. et al. *The Annotated Transformer*. 2018. Disponível em: <http://nlp.seas.harvard.edu/annotated-transformer/>.
- [4] HARVARD NLP. *The Annotated Transformer*. GitHub repository, 2018. Disponível em: <https://github.com/harvardnlp/annotated-transformer/>.
- [5] DEVLIN, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2018.
- [6] RAFFEL, C. et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2020.

Aspecto	BERT	T5
Arquitetura	Apenas encoder	Encoder-decoder
Objetivo principal	Compreensão	Geração e transformação de texto
Treinamento	MLM	Text-to-text
Uso típico	Classificação, QA, NLI	Tradução, resumo, geração
Custo de inferência	Menor	Maior

Tabela 1: Comparação resumida entre BERT e T5.